

DPS: Dual-Mode Precision LLM Serving with Semi-Unified Memory

Abstract—Existing LLM serving systems virtualize and optimize KV-cache memory, but treat model-weight memory as fixed throughout execution. Recent work on multi-precision model representations challenges this design by allowing a single stored model to support both full-accuracy and lower-precision execution, making the effective weight footprint runtime-dependent. This creates an opportunity under bursty workloads, where temporary spikes in KV-cache demand often determine throughput and SLO compliance. We present DPS, a dual-precision LLM serving system that turns weight memory into an elastic resource: under normal load, DPS serves the full-accuracy model; under KV pressure, it switches to a nested, lower-precision variant and repurposes unused weight memory for KV cache blocks. DPS is built on Semi-Unified Memory (SUM), which partitions the weight region into a persistent lower-precision sub-region and a shared region that alternates between residual weight tensors and KV-cache blocks, preserving compatibility with paged KV-cache management. Experiments across diverse workloads show that DPS reduces preemptions and improves throughput and SLO compliance over static-allocation baselines.

I. INTRODUCTION

Transformer-based large language models (LLMs) such as GPT [1], Llama [2], Qwen [3], and DeepSeek [4], [5] have been emerging as foundation components in modern AI services, including virtual assistants, chatbots, text, image, and code generation [6], [7]. Their strong performance across various domains has attracted significant attention and driven significant user demand. To improve throughput and system efficiency, existing LLM serving systems, such as Orca [8], vLLM [9], [10], and SGLang [11], [12], employ various optimization techniques, including continuous batching [8], [9] and memory-efficient attention mechanisms such as PagedAttention [10]. In particular, inspired by classical virtual memory and paging techniques, vLLM facilitates flexible sharing of KV (key-value) cache within and across requests to effectively reduce memory usage and thereby achieves high utilization in KV cache memory [9], [10]. These optimizations highlight the importance of memory management, especially in managing the KV cache, in achieving high system performance.

Modern LLM inference operates in two stages: a *prefill* stage that processes the input prompt and a subsequent *decode* stage that generates tokens. The prefill stage is dominated by compute-intensive general matrix multiplication (GEMM) operations, which can be effectively accelerated on GPUs [16], [17], [15] and Tensor Processing Units (TPUs) [18], [19]. On the other hand, the decode stage is dominated by memory-intensive general matrix-vector multiplication (GEMV) or attention operations, which under-utilize the GPU computing resources. LLM-serving systems such as Orca [8], vLLM [9],

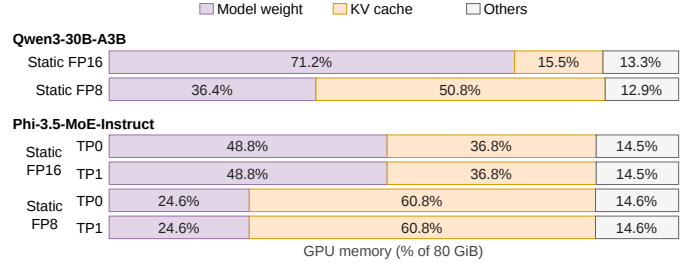


Fig. 1: Memory layout when serving Qwen3-30B-A3B [13] and Phi-3.5-MoE-Instruct [14] on NVIDIA H100-80G GPUs [15].

[10], and SGLang [11] enhance throughput by batching multiple requests together [8], [9], but the effective batch size is limited by the available GPU memory to store all the KV vectors of the active requests.

One fundamental challenge in LLM-serving systems is effectively managing GPU memory when dynamic workloads require different degrees of memory capacity for the KV caches [9]. Popular LLM-serving systems such as vLLM [9], SGLang [12], TensorRT-LLM, TGI [20], llama.cpp, and DeepSpeed-Inference statically allocate memory for model weights and KV cache. Once the sizes of these memory spaces are determined, they will not be changed during the runtime. For example, Figure 1 illustrates the memory distribution for a 30B-parameter LLM on an NVIDIA H100 GPU with 80GB RAM [15]. 71.2% of the GPU memory is statically allocated for the model weights during serving. 15.5% of the memory is used to store the dynamic states of the requests, which include the key and value tensors associated with the attention mechanism, commonly referred to as KV cache. Since the model weights are assumed to be constant and the activations only occupy a small fraction of the GPU memory, the way the KV cache is managed is critical in determining the maximum batch size [9]. When request traffic surpasses the system’s capacity, LLM serving systems like vLLM [9], [10] must prioritize a subset of requests and may need to preempt some requests, for example, the latest requests due to their first-come, first-served (FCFS) scheduling policy.

As a promising alternative to FP16, 8-bit floating-point formats (FP8) are gaining traction in LLM serving, offering up to 2× higher peak throughput and smaller memory footprint at a slight accuracy degradation [21], [22], [23], [24]. Interestingly, an FP8 model can be directly derived from the corresponding FP16 model and later restored to FP16 to recover the accuracy [24]. While conventional quantization methods do

not allow efficient use of both high (unquantized) and low (quantized) precision, this property opens the opportunity of having an elastic KV cache memory: the memory occupied by the residual precision (the difference between FP8 and FP16) can be used as a shared resource between KV FP16 weights and the KV cache. For instance, reconsidering the above case of serving a 30B model on an 80G GPU. With FP8, the model requires only 36.4%, leaving 50.8% for the KV cache, as illustrated in Figure 1. Similar patterns are also observed on Phi-3.5-MoE-Instruct [14] when it is served with two GPUs using tensor parallelism (TP).

We present DPS, a dual-precision serving mechanism for LLM serving systems to exploit this opportunity. Under normal loads, DPS serves the **FP16-precision model (referred to as full-quality mode)** with performance comparable to that of a conventional serving system. When DPS identifies high KV pressure (request bursts or long-context inputs), DPS switches to **FP8 model (referred to as performance mode)**, which is directly derived from the FP16 model, and dynamically reallocates the freed weight memory as additional KV blocks. When the KV pressure subsides, DPS restores the weights to gradually return the system to function at **full-quality** mode.

The paper’s contributions are summarized as follows.

- **Semi-unified memory (SUM).** We introduce a memory abstraction that partitions the original model weight memory into a persistent **lower-precision region and a shared region**. The shared region can store either the residual weights in **full-quality** mode or KV cache blocks in **performance** mode. Our implementation is based on CUDA Virtual Memory Management to preserve compatibility with the default KV cache allocation in vLLM.
- **Dual-precision execution with asymmetric switching.** DPS exploits the trade-off between FP8 and FP16 to balance accuracy and performance. Switching between these two precisions is asymmetric: transitioning from FP16 to FP8 requires no data movement, while the reverse requires Host-to-Device communication. We design a background restore mechanism that fully overlaps this communication cost with the ongoing FP8 computation, enabling smooth transitions.
- **SUM-aware scheduling policy.** We integrate SUM with vLLM’s scheduler. The scheduler actively monitors the KV cache pressure and determines when to switch to prevent cache thrashing and mitigate accuracy loss.
- **Implementation and evaluation.** We implement DPS on top of vLLM and evaluate it across both dense and MoE models and various production workload traces. Our results show that DPS improves sustained throughput by 2.1–3.3 $\times$ and effective pass@1 by up to +41 pp over Static FP16, while preserving FP16-class accuracy.

II. BACKGROUND

A. LLM architecture and LLM serving systems

LLM architecture. Modern LLMs typically consist of stacked decoder blocks [1], [2], [25]. Each block is further de-

composed into self-attention (SA) and feed-forward (FFN) layers. Recent architectures introduce Mixture-of-Expert (MoE) layers [26]. The *Self-Attention* module projects the hidden states of the previous layer into query (Q), Key (K), and value (V) vectors, and computes attention scores that represent the context-awareness of each token. In autoregressive inference, to avoid redundant recomputation, self-attention relies on the K and V vectors computed from the previous steps. LLM inference systems generally store these KV vectors on GPU memory to reduce the load latency. This memory cost becomes the performance bottleneck as the number of required KV vectors increases with the number of in-flight requests. Insufficient KV cache capacity may require the system to use a high-latency memory storage or preempt requests. These activities degrade performance and may cause the system to run at low utilization or fail to meet service-level objectives (SLOs).

LLM serving systems. LLM serving systems, such as vLLM [9] and SGLang [11], focus on improving throughput and resource efficiency via various optimization techniques in scheduling and memory management. One essential optimization is continuous batching, which groups incoming requests into single batches to improve hardware utilization. vLLM introduces an advanced KV cache management algorithm, called PagedAttention or vLLM [9]. PagedAttention is inspired by virtual memory systems to enable flexible memory allocation and to share KV blocks between tokens and requests. These techniques also reduce memory fragmentation. SGLang uses prefix-aware caching and reuse mechanism [11] to manage the KV cache. Despite these advances, most systems statically partition the memory space between the weight and KV caches at initialization time. Once allocated, the partitions remain fixed during the entire serving time. This design limits the flexibility to adjust the memory partitions in the presence of workload variation.

B. FP16/FP8 for LLMs

Unlike in computer vision tasks, LLMs often exhibit activation outliers [27], [28], [29], [30], making floating point formats (e.g., FP16 and FP8) widely used. A floating point format with x exponent bits and y mantissa bits is denoted as $ExMy$. For instance, FP16 is also referred to as E5M10, where 5 and 10 bits are used for the exponent and mantissa, respectively. Two emerging FP8 formats, E5M2 and E4M3, are defined similarly. While FP16 has become the de facto standard for LLM serving, FP8 is attracting significant attention with two fundamental factors: (1) the representational advantages of floating-point formats to capture outliers, and (2) the increasing hardware support for FP8 arithmetic. Recent studies [21], [22], [23], [24] have demonstrated that FP8 is able to maintain a high accuracy with a small degradation, compared to FP16. For example, FP8 (E5M2) has the same number of exponent bits as FP16, while it has a lower precision than FP16 due to a smaller number of mantissa bits. More importantly, FP8 reduces the bandwidth requirement by up to 2 $\times$, and FP8 is natively supported in arithmetic units by many

modern hardware systems, including NVIDIA Hopper [15], Intel Gaudi HPUs, or NPUs [31].

C. GPU Memory Characteristics

Modern GPUs feature a hierarchical memory system, including high-bandwidth memory (HBM), on-chip caches (L1, L2, shared memory), and register files. The latency of accessing HBM is much higher than that of on-chip caches. However, on-chip cache sizes are much smaller. Therefore, optimizing LLM operations mostly involves fitting the working set into on-chip and register memory. During LLM inference, the pre-fill phase benefits from compute-intensive GEMM operations that efficiently utilize NVIDIA Tensor Cores (Matrix Cores for AMD GPUs). However, the decode phase is dominated by memory-bound operations that require frequent HBM access. This results in low arithmetic intensity and GPU utilization. As a result, optimizing memory usage and data movement is critical for improving overall system performance. In particular, reducing memory footprint or increasing effective memory capacity can directly translate into higher throughput.

III. MOTIVATION

A. Challenge: Dynamic Load in LLM Serving

Bursty LLM workloads. LLM serving systems face highly dynamic and bursty computing patterns due to substantial variability in the request arrival rates (i.e., request bursts), input prompt, and output sequence lengths across requests [32], [33]. As illustrated in Figure 2(a), the production workloads of the Microsoft Azure LLM services and BurstGPT reveal rapid fluctuations: the request rate exhibits nearly a fivefold variation between the lowest and highest load periods. In addition, these fluctuations occur with high temporal frequency, indicating second-level variability. These fluctuations pose a practical challenge for real-world LLM inference workloads, which are not fully captured in settings such as vLLM [9]. An important observation is that the periods of high KV cache pressure are not permanent; they are usually interleaved with low-pressure periods. This transience has two implications. First, static solutions that constantly trade quality for performance (e.g., always using FP8 or static weight offloading), or vice versa, are suboptimal because they incur a permanent cost for an intermittent problem. Second, rapid fluctuations demand a lightweight switching mechanism, which in turn requires an efficient system design. This observation motivates a fast, runtime-adaptive memory management mechanism that can elastically respond to different workload characteristics.

Long TTFT/TPOT and SLO violation. A request burst may cause a long Time-to-First-Token (TTFT) latency and SLO violations, as illustrated in Figure 2(b). As the system load increases, even small surges can cause sharp spikes in TTFT latency. Specifically, a serving system quickly exceeds the SLO threshold once GPU memory becomes insufficient to schedule new requests for prefilling or to continue decoding for the ongoing batch. Consequently, an incoming request is forced to wait until memory is reclaimed, incurring significant queuing latency with SLO violation. The problem is

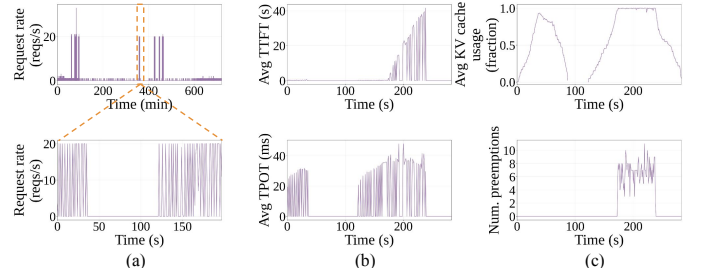


Fig. 2: (a) Workload Fluctuations, (b) TPOT/TTFT, SLO (c) KV cache usage, preemptions.

exacerbated by long-context requests, especially in modern applications, where the context length can reach 32K-1M tokens [34], [35], [36].

KV Cache Pressure and Preemptions. The above issue can also be visualized via KV cache utilization or pressure. As the system load increases, the KV cache memory is highly utilized. When the request traffic surpasses the system’s capacity, the serving systems like vLLM [9], [10] must prioritize a subset of requests and need to preempt some requests. As illustrated in Figure 2(c), when the system load increases with high fluctuations, the KV cache memory is nearly full, and preemptions occur more frequently.

Preemptions are particularly costly for long-context workloads. A preempted request incurs recomputation of the entire KV cache from scratch at the rescheduling point, or swapping all relevant KV cache items between GPU and CPU memory. Either way, the cost is proportional to the input context length. For a coding agent processing a 64K-token repository context, preemption can add seconds of latency that generally exceed SLO budgets. Lastly, preemption prevents current LLM-serving systems such as vLLM [9] and SGLang [12] from realizing the full potential of GPU utilization. As the context length continues to grow, driven by both user demand and model support, the preemption cost increases proportionally, making preemption-avoidance solutions more attractive for maintaining the SLO.

B. KV Cache Pressure and Preemption Mitigation via Dual-Precision Serving (DPS)

Opportunity. *Dual-precision serving* presents an opportunity to mitigate KV cache pressure. For example, consider a memory layout for serving a 13B model on an A100-40G GPU. When applying a nested FP8/FP16 model [24], 13 GB, 13 GB, and 12 GB are primarily allocated for upper and lower FP8 weights and KV cache, respectively, accounting for 95% of the GPU memory. However, under a large number of requests, the serving system performs a **performance** mode that uses only 13 GB of upper FP8 weights for computation, thereby improving system performance. Our observation is that 13 GB of the lower FP8 weights can be deallocated and remapped to the KV cache. As a result, in this particular case, the KV cache size can be elastically extended by 108.33% ($= 13/12$), significantly mitigating KV cache pressure, and

thereby improving system throughput and SLOs. Meanwhile, when all 26 GB are used for the upper and lower FP8 weights, this preserves the behavior of an LLM serving system, such as vLLM [9], under a **full-quality** mode.

Challenge. The above *dual-precision serving* presents several challenges. Firstly, serving an LLM in **full-quality** mode with both upper and lower weights should incur negligible overhead compared with a baseline system, such as [9]. Secondly, extending the KV cache size, for example, from 12GB to 25GB, must preserve the powerful page KV cache management in existing LLM serving systems. This requires a *lightweight accuracy-to-performance* mode switching and a virtual memory scheme to elastically extend the KV cache. Lastly, restoring a **full-quality** mode from a **performance** mode intuitively requires loading 13 GB of lower FP8 weights from the host to GPU memory, which may cause considerable latency during serving. This poses a critical challenge to design an effective mechanism for a *smooth performance-to-accuracy* mode switching.

IV. METHOD

A. Design Methodology

Persistent and Dynamic Memory. This subsection revisits the fundamental concept of memory management in LLM-serving systems, such as vLLM [10]. Specifically, inspired by a traditional virtual memory mechanism in an operating system, KV blocks are typically managed in pages and dynamically (de)allocated during serving. Notably, a model’s parameters are typically assumed to be preloaded to GPU memory before serving and considered **persistent during serving**. For example, FP16 and FP8 models are loaded into a persistent region, as illustrated in Figures 3(a) and (b), respectively. This assumption is intuitive because a typical LLM model is large, making it costly to transfer an entire model from CPU memory to GPU memory at runtime. Meanwhile, a fundamental observation is that if LLM inference can be performed with a partial model during serving, KV memory can be temporarily extended, possibly relaxing KV memory pressure and thereby improving system throughput. Intuitively, instead of being persistent, weight memory may be elastic, with parts of it mapped to KV blocks at runtime.

Dynamic Networks and a Nested Model. Under bursty workloads, KV cache pressure and a demand for a KV cache extension may occur instantly, which becomes difficult to predict. To effectively handle such an instant KV pressure, a system must quickly switch from a persistent weight memory to a temporarily elastic one, leaving more space for KV blocks. Dynamic networks, including nested FP [24], any precision networks [37], [38], early-exit networks [39], [40], [41], provide a promising opportunity to address the challenge. Without a loss of generality, let’s consider an FP16 model (e.g., E5M10) storing both an FP8 model (e.g., E5M2 or a custom E4M3 format [24]) and a residual model, as illustrated in Figure 3(c). More specifically, an FP16 model is split into two parts: **a FP8 model (e.g., with upper (or MSB) weight tensors)** stored in a persistent region and a residual model (e.g.,

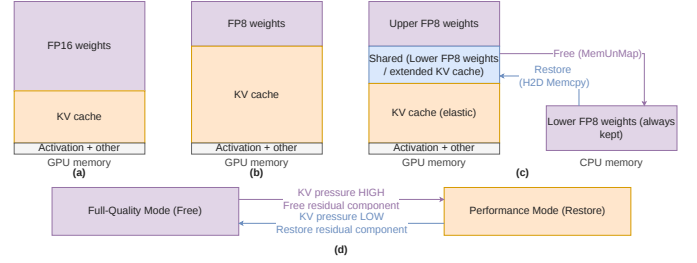


Fig. 3: Three memory layouts: (a) FP16 model, (b) FP8 model, and (c) Proposed semi-unified memory (SUM) with a dynamic switching between a **full-quality** mode and a **performance** mode (d).

with **lower (or LSB) weight tensors** stored in a shared region. This can facilitate quick switching between FP16 and FP8 inference, temporarily freeing the residual model’s memory for KV blocks and thereby boosting system performance.

Dual-Precision Serving (DPS) Overview. Inspired by the aforementioned observations, this subsection presents an overview of DPS, a dual-precision serving mechanism, which can be directly integrated into existing LLM serving systems, such as vLLM [10]. DPS introduces a novel semi-unified memory (SUM) scheme. SUM is a memory abstraction that allows a shared physical region to dynamically back **two distinct virtual tensors**: one for the residual weights, the other for the KV cache pool. In the following subsection, we first discuss the SUM architecture and asymmetric mode switching. Next, dual-precision execution is explained in detail. Lastly, we provide a theoretical performance bound for a DPS problem.

B. Mapper: Semi-Unified Memory Abstraction

SUM is composed of four main components: a persistent region holding the lower-precision representation of model weights, a shared region whose physical memory may back either residual high-precision weights or KV cache blocks, a CPU-side residual backup used for restoring residual weight data after it has been overwritten, and a coordinator, called the SUM Manager, that owns the binding state of the shared region and exposes allocation operations to the serving system.

Persistent Region. The persistent region holds the **lower-precision representation** of model weights. Specifically, the region can be used to store an FP8 model, as illustrated in Figure 4(a). It is allocated via the standard PyTorch allocator at deployment startup and occupies a fixed portion of GPU memory throughout the deployment’s lifetime. Notably, this retains the structure of the conventional serving systems like vLLM [10]: *the system can always execute a model, for example, an FP8 one, regardless of the state of the shared region or KV cache*. For an FP8 model, one can directly obtain an FP8 tensor in E5M2 format from its corresponding FP16 (E5M10) tensor by using all five exponent bits and keeping the two MSB bits in the mantissas. In a specific case, if the range of data is small and the first MSB bit in the exponent is zero, one can obtain an FP8 tensor with an E4M3 format by keeping four out of five exponent bits and three MSB mantissa bits,

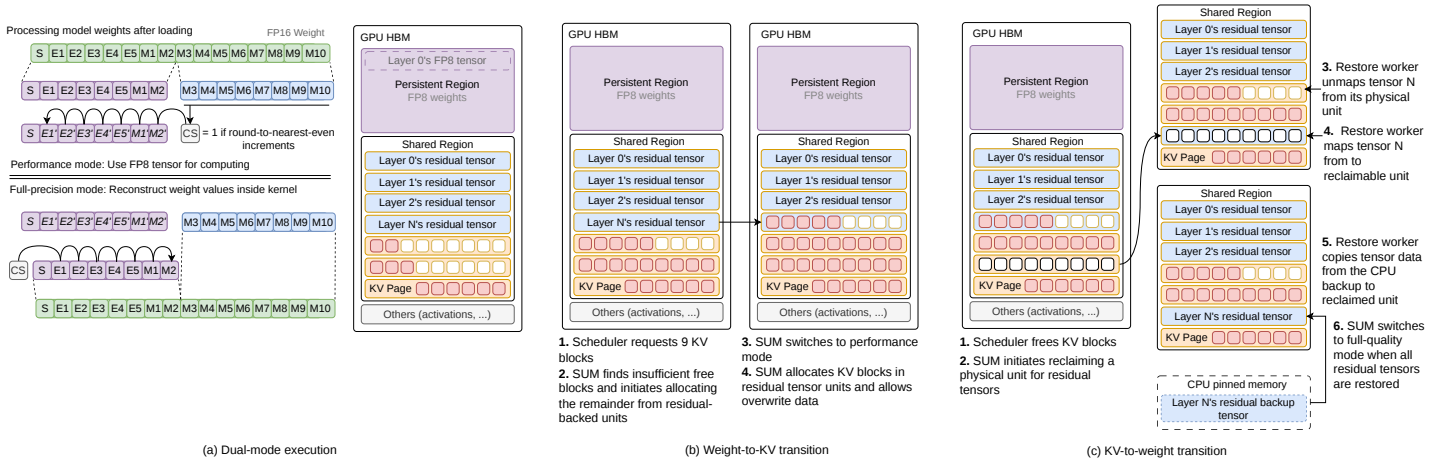


Fig. 4: Dual-mode precision with SUM: (a) SUM mapping for nested FP16-FP8 serving, (b) Weight-to-KV transition, and (c) KV-to-weight transition.

as illustrated in Figure 4(a). This has been exploited in [24], as many layers in LLMs have a small weight range.

Shared Region. The shared region is the elastic component of SUM. It is constructed from two primitives: *physical units* and *virtual tensors*. A physical unit is a contiguous region of GPU memory configured via CUDA Virtual Memory Management (VMM) API [42], [43]. A virtual tensor is a contiguous virtual address range exposed to the serving system through standard tensor access patterns. CUDA VMM separates virtual address reservation, physical handle allocation, and mapping between them, allowing a single physical handle to be mapped into multiple virtual addresses simultaneously and remapped at runtime. SUM relies on this separation to maintain dynamic bindings between physical units and virtual tensors.

It is essentially noted that SUM, in particular, the shared region, builds a bridge between a *persistent* model and a dynamic *page-based KV cache* memory in a serving system, such as vLLM [10], as demonstrated in Figure 4. When the *physical shared memory is entirely allocated to page-based (virtual) KV blocks*, this facilitates serving an FP8 model stored in the persistent region. This refers to a **performance** mode in which the KV cache can fully utilize shared memory, thereby possibly reducing KV cache pressure and preemptions and boosting performance. Obviously, it can further increase system throughput by leveraging FP8 computing, e.g., halving memory accesses by 2x or doubling peak tensor-core throughput with hardware support [24], [15]. Meanwhile, *the shared region can be overlaid and assigned to a partial persistent model*. For example, as described in Figure 4(a), the shared region can be configured to store a residual FP8 model that is combined with the FP8 model in the persistent region to form an FP16 model. This facilitates a **full-quality** mode because an entire FP16 model becomes persistent on GPU memory, like in the conventional vLLM [10] with FP16 model execution. Essentially, SUM provides a wrapper for a *persistent* model and a dynamic *page-based KV cache* in a serving system, while retaining effective KV cache management as in existing

serving systems, such as vLLM [10].

Residual Backup. The residual backup is a pinned, CPU-side copy of all transformer layers’ residual weight tensors, initialized once at deployment startup. It serves as *the source for restoring residual weights after physical units have been overwritten by KV cache writes*. More specifically, after the physical shared memory is entirely assigned to page-based (virtual) KV blocks, the residual weights on GPU memory is lost and overwritten by KV. When KV blocks are freed, leaving a sufficient space in the shared region for a residual weight, the residual backup provides a source for reloading residual weights from CPU into the shared region on GPU.

SUM manager. The SUM Manager maintains the binding state of all physical units in the shared region and records the current mapping between physical units and virtual tensors. It processes allocation and deallocation requests for KV cache blocks issued by the serving system. When an allocation request targets units that hold no residual data, the SUM Manager *returns the requested blocks directly*. When a request targets a unit holding a residual-weight virtual tensor, the SUM Manager *signals the precision-mode transition* to the model executor and marks the affected residual mapping as stale.

The SUM Manager observes deallocation events from the serving system to identify physical units whose KV cache blocks have all been freed. It enqueues a restore task for each such unit, and a background worker thread internal to the SUM Manager processes the queue: for each task, the worker *selects* a stale residual weight virtual tensor, unmaps it from its previous unit, maps it to the reclaimable unit, and copies the layer’s residual weight data from the residual backup into the unit. Once all layers’ residual weight data have been restored, the SUM Manager signals the model executor to *transition back to full-precision execution*. Performing restore work in the background keeps VMM operation latency off the allocation and deallocation hot path.

Dual-mapping property. An alternative design would unmap each physical unit before rebinding it to a different virtual

tensor, avoiding the dual-mapping property entirely. Such a design is incompatible with vLLM’s asynchronous scheduling model. In this model, the scheduler may mark KV blocks as free while GPU workers still hold in-flight reads issued against those blocks by earlier submitted work. If freeing a KV block involved unmapping its backing unit, the in-flight reads would access unmapped memory, causing an illegal memory access error. Resolving this race would require synchronizing the scheduler against the workers on every free event, which defeats the purpose of asynchronous scheduling. SUM avoids the race by construction: the KV virtual tensor is **permanently mapped to every physical unit** in the shared region, so marking a unit’s residual mapping as stale does not affect the KV mapping, and in-flight reads through the KV path remain valid regardless of residual state.

C. Scheduler: Dual-Precision Execution

Overall concept. SUM facilitates non-stop execution and can be integrated in vLLM, as illustrated in Figure 5. Specifically, at runtime, the model executor follows one of two paths depending on the chosen mode. In a **full-quality** mode, it combines the **FP8 weight tensor from the persistent region with the residual weight tensor from the shared region** to reconstruct the original FP16 values during computation. In a **performance** mode, only the FP8 weight tensor is used; the residual weight tensor in the shared region is not accessed, and its physical memory may at this point be holding KV cache data rather than residual weight data. The SUM Manager assigns a current precision mode to each worker, and the model executor reads that mode at the start of each forward pass. This mode is set to a **performance** mode when the SUM Manager processes an allocation request that overwrites a residual-bound unit, and reset to a **full-quality** mode when the background restore worker has populated all residual weight tensors. Mode transitions are batch-aligned: the SUM Manager updates the mode only between forward passes, and the model executor executes each pass entirely in a single mode. Obviously, SUM also enables single-mode execution.

Dual-mode serving (DPS) with asymmetric switching. This subsection unleashes the power of SUM for dual-precision serving. Notably, runtime switching for LLM serving with multi-precision representations has recently been discussed [24], [38], [44]. DPS implements **GroupedGEMM kernels** that perform NestedFP-style reconstruction [24], [21], supporting dense LLMs and MoE models. DPS extends the encoding scheme with an alternative **FP8 E5M2** representation for layers whose weight magnitudes fall outside the range that E4M3 can express. With this extension, all projection layers in the model are eligible for FP8 execution, and the **per-layer encoding is selected offline at model loading time based on weight statistics**. Inspired by [24], SUM facilitates a nearly-zero-cost switching from a **full-quality** mode to a **performance** mode, as described in Figure 4(b). Specifically, because the FP8 model is in the persistent region, the system can execute it directly without stalling while loading a new model. It is especially effective for bursty workloads

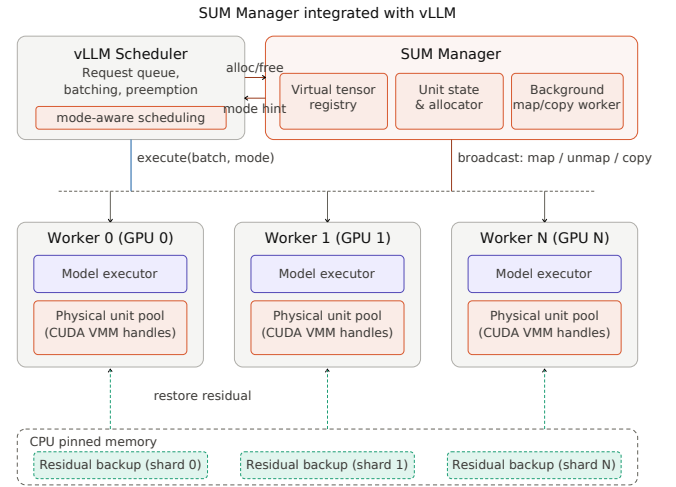


Fig. 5: Dual-precision serving architecture with SUM backend.

because the demand for a **performance** mode appears instantly and non-deterministically. More importantly, unlike [24], the shared memory for the (previously persistent) residual weight is unmapped and reallocated to KV cache blocks, effectively mitigating high KV cache pressure and potentially reducing preemptions.

SUM also facilitates a restoration of a **full-quality** mode, as illustrated in Figure 4(c). Restoration is the process of bringing the residual weight (the lower FP8 byte) back from CPU memory to GPU memory so that subsequent forward passes can run in a **full-quality** mode. We use a dedicated non-blocking copy stream to perform the required Host-to-device (H2D) memory copy and to overlap it with the ongoing computation. Since GPUs generally have a single PCIe copy engine, long H2D copies can stall the entire decoding pipeline. To avoid this problem, we schedule the **copies in small chunks** and let **other system H2D copies take place** between the chunk copies. This fragmentation deliberately leaves the engine between chunks so that small H2D traffic from the request scheduler can interleave without significant delay. At engine startup, when many copies are issued back-to-back and no other PCIe traffic is present, the system uses a simpler one-shot path: each copy is a single asynchronous transfer, terminated by a CUDA event that the compute stream waits on. Chunking is only reserved for the runtime restore path. Empirically, we observe that a fixed chunk size of 4MiB works well for most cases. Notably, because H2D transfers overlap with ongoing inference and there is no synchronous wait that pauses the engine, the **actual cost paid is therefore not the duration of the transfer but the slight PCIe contention it imposes on per-step scheduler uploads**, which the chunked yield design is meant to bound.

Granularity of the precision switch. Notably, restoration cannot itself preempt or replay a request; once a request’s decode has produced a token under single-weight precision, the token is final, and only subsequent decode steps are

Algorithm 1 KV block allocation and deallocation in SUM

```
1: FREE: units w/ no KV blocks and no residual mapping.
2: BACKED: units bound to residual weight virtual tensors
3: AVAIL, FULL: KV units with at least one free block or with no free blocks
4: STALE: invalid residual mappings awaiting restoration
5: EVICTED: weights whose residual data is on CPU only, awaiting restoration
6: procedure ALLOCKVBLOCK( $n$ )
7:   result  $\leftarrow \emptyset$ 
8:   while |result| <  $n$  do
9:     if AVAIL is empty then
10:       if FREE is non-empty then
11:          $u \leftarrow \text{pop from FREE}$ 
12:       else if BACKED is non-empty then
13:          $u \leftarrow \text{pop from BACKED}$ 
14:         activate lower-precision mode
15:         mark  $u$ 's residual mapping as stale
16:       else
17:         return failure
18:       end if
19:       add  $u$  to AVAIL
20:     end if
21:      $u \leftarrow \text{pop from AVAIL}$ 
22:     allocate up to  $n - |\text{result}|$  KV blocks from  $u$ ; append to result
23:     if  $u$  has no free blocks then add  $u$  to FULL
24:     else add  $u$  to AVAIL
25:     end if
26:   end while
27:   return result
28: end procedure

29: procedure FREEKVBLOCK(blocks)
30:   for each unit  $u$  containing a block in blocks do
31:     mark the corresponding blocks free in  $u$ 
32:     if  $u$  was in FULL then move  $u$  to AVAIL
33:     end if
34:     if  $u$  has no allocated blocks then
35:       remove  $u$  from AVAIL
36:       add  $u$  to FREE
37:     if EVICTED is non-empty then
38:       wake RESTOREWEIGHTS
39:     end if
40:   end for
41: end procedure
```

affected by the restored precision. The system pre-captures **two CUDA graphs** per supported batch size: one with single-weight (FP8-only) and the other with dual-weight (FP8+residual) kernels. The dispatcher picks exactly one for each forward pass. As a result, a precision change takes effect at scheduling-step boundaries, never within a step: every layer in a given forward pass runs at the same precision. A request that spans a mode transition continues without a restart: tokens generated before the flip use FP8, and subsequent tokens use FP16, but the request itself is never recomputed.

D. KV-Cache-Pressure-aware DPS

Precision-switching Algorithm. This subsection presents a DPS algorithm based the KV cache pressure. The central idea is to build a KV block allocation and deallocation/free with SUM, as summarized in Algorithm 1. This is based on runtime internal states of SUM, including FREE, BACKED, AVAIL, FULL, and STALE. Specifically, FREE stores *free* physical units which are not assigned to either a KV block nor a residual

weight's block. BACKED save units bound to residual weight virtual tensors. While AVAIL marks KV units with at least one free block, FULL indicates KV units with no free blocks. STALE marks invalid residual mappings awaiting restoration.

The KV block allocation is presented in lines 6-28 of Algorithm 1. Consider a request to allocate n new KV blocks and let u be a temporary list to store physical units for these KV blocks. The procedure starts with checking if AVAIL is empty. If there are no KV units with at least one free block (line 9), it first tries to find an empty allocation slot in FREE (lines 10-11). Notably, this step actually retains the effective KV management in a serving system, such as vLLM [10]. The key idea is that if there is an unused slot for KV-cache, it reserves it for a new KV block by updating the allocation slot list u . Meanwhile, if there is no available slot and this is under a **full-quality** mode (e.g., non-empty BACKED) (line 12), it updates u from BACKED (line 13) and marks the residual mapping of u as stale (line 15). Also, it activates a **performance** mode (line 14). This step clearly demonstrates that can instantly switch from a **full-quality** mode to a **performance** mode to resolve KV cache pressure (e.g., empty FREE). Lastly, the allocation slot list u is used to allocate KV blocks (lines 21-25).

The KV block deallocation is shown in lines 29-42 of Algorithm 1. Consider a request to free *blocks*. For each unit u containing a *target-to-free* block, mark the free block in u and examine existing states FULL, AVAIL, and FREE. If u is currently in FULL, move it to AVAIL (line 32). If there is no allocated block u , remove it from AVAIL and add it to FREE (lines 35-36). If it is a **performance** mode and there is sufficient memory space in the shared region to accommodate a residual weight (e.g., non-empty EVICTED) (line 37), it wakes a state of weight restoration (e.g., RESTOREWEIGHTS).

Performance Bound on the Switching Heuristic. Our precision-switching scheme can be modeled by a standard discounted Markov Decision Process. The state is (k, m) , where $k \in \{0, \dots, C_1\}$ is the KV-cache occupancy and $m \in \{0, 1\}$ is the precision mode (0=FP16, 1=FP8). The action is whether *stay* or *switch*, and the per-step cost combines latency, accuracy, and a fixed switching cost β . Because FP8 enlarges the KV capacity, its advantage in reducing preemption is non-decreasing in k . This is the monotonicity condition [45] that implies the optimal value function V^* is submodular in (k, m) and with any thresholds (κ_L, κ_H) that our algorithm picks, the induced policy $\hat{\pi}$ satisfies the standard policy-search bound [46].

$$V^{\hat{\pi}}(s_0) - V^*(s_0) \leq \frac{\mathcal{E}(\hat{\pi})}{1 - \gamma},$$

where $\mathcal{E}(\hat{\pi}) \geq 0$ is the per-step *miscalibration cost* of the chosen thresholds—the cost penalty for switching at a slightly-wrong KV occupancy—and is zero when the thresholds are placed at the indifference point at which the marginal benefit of switching exactly balances the switching cost β .

TABLE I: Offline benchmark accuracy among BF16, FP8, and DPS with FP16-only and FP8-only. The rightmost column reports the mean per-benchmark difference relative to BF16 (in percentage points). Per-benchmark variation of $\pm 1\text{--}2\text{pp}$ is within the noise floor of these benchmarks.

Model	Method	BBH	GPQA	MATH-500	MMLU Pro	MUSR	IFEval	LiveCodeBench	Avg Δ
Phi-3.5-MoE	BF16	74.00	36.38	37.00	60.09	45.90	64.88	22.84	—
	FP8	76.27	35.94	38.80	59.66	45.77	42.70	21.80	−2.88
	DPS full-quality mode	73.91	35.94	36.20	60.31	46.03	66.73	23.70	+0.25
	DPS performance mode	74.11	35.49	37.60	59.73	47.35	65.62	22.94	+0.25
Qwen3-30B-A3B	BF16	56.40	43.53	75.80	72.42	42.46	81.70	56.21	—
	FP8	57.32	44.87	73.40	72.54	41.27	82.07	56.02	−0.15
	DPS full-quality mode	58.27	41.74	75.40	72.34	41.93	81.89	55.07	−0.27
	DPS performance mode	58.81	42.63	74.00	72.37	42.86	82.44	57.25	+0.26
GLM-4.7-Flash	BF16	81.25	37.72	20.60	63.49	41.80	80.41	34.88	—
	FP8	80.89	33.48	22.80	62.80	42.20	79.85	34.50	−0.52
	DPS full-quality mode	81.74	34.15	18.20	63.44	42.06	77.63	34.88	−1.15
	DPS performance mode	81.65	36.83	21.60	63.53	44.31	75.97	35.07	−0.17

V. EVALUATION

A. Methodology

We implement DPS on top of vLLM [9] 0.18.0 and evaluate it on three MoE models — Phi-3.5-MoE [47], Qwen3-30B-A3B [13], and GLM-4.7-Flash [48] — running on H100 80GB GPUs [15]; we use tensor parallelism (TP) when a model exceeds a single device. Specifically, Phi-3.5-MoE uses TP=2. To emulate bursty serving, we replay two production traces, BurstGPT and Azure: from each we sample a contiguous 2,000-request window and scale the arrival rate by $200\times$. Quality is measured on seven benchmarks — LiveCodeBench (live coding) plus MMLU-Pro, BBH, GPQA, MATH-500, MUSR, and IFEval (knowledge, reasoning, and instruction following).

Baselines. We compare DPS against three static-precision configurations — BF16 (native), static FP8 (symmetric static per-channel weights and symmetric dynamic per-token activations [49], [27], [23]), and NestedFP — plus three baselines that relieve KV pressure through orthogonal mechanisms, all implemented in vLLM [9]. KV-OFFLOAD evicts least-recently-used KV blocks to pinned CPU memory once the GPU KV pool is exhausted and reloads them on demand. KV-QUANT stores keys and values in FP8, enlarging the effective KV pool at the cost of dequantization on the read path. WEIGHT-OFFLOAD keeps a fraction of the weights in pinned CPU memory and streams them in during each forward pass, freeing HBM for additional KV blocks; we use vLLM’s offloading mechanism with the per-model configuration that yields the best performance.

SLO definition. Following Sarathi-Serve [50], T_{base} is the P99 decode iteration time for a batch of 32 requests with 4k context and no prefill interference. We measure it on the FP16 configuration, which is DPS’s base mode before any fallback to FP8, so the SLO is consistent across all systems: $T_{\text{base}} = 19.74, 18.19, \text{ and } 23.03\text{ ms}$ on Phi-3.5-MoE, Qwen3-30B-A3B, and GLM-4.7-Flash, respectively. A request satisfies the joint SLO iff (1) its maximum intra-request time-between-tokens (TBT) is $\leq 5 \times T_{\text{base}}$ and (2) its time-to-

first-token (TTFT) is $\leq 2000\text{ ms}$. Following DistServe [51] and SLOs-Serve [52], system capacity is the highest sustained mean rate at which $\geq 90\%$ of requests satisfy this SLO.

Metrics. Accuracy is task quality on LiveCodeBench and MMLU-Pro; performance is serving behavior under the accelerated BurstGPT [33] and Azure traces. Because conventional benchmarks score model quality in isolation and ignore latency, we also report *effective pass@1* [51], [53], [54], the fraction of submitted requests that both satisfy the joint SLO and produce a correct answer per the benchmark’s standard pass@1: $\text{effective pass@1} = \text{SLO attainment} \times \text{pass@1}_{\text{served}}$. Requests that miss the SLO contribute zero regardless of accuracy.

B. Experimental Results

1) *Accuracy:* Table I reports offline benchmark scores for three models under four configurations: BF16 (native precision), static FP8, DPS **full-quality** mode (FP16 reconstructed from the NestedFP decomposition), and DPS **performance** mode (FP8 component only, with non-NestedFP layers retained at original precision). The rightmost column gives the mean per-benchmark deviation from BF16 in percentage points (pp).

Both DPS modes preserve BF16 accuracy. **full-quality** mode shows mean deltas of +0.25, −0.27, and −1.15pp on Phi, Qwen, and GLM, while **performance** mode shows +0.25, +0.26, and −0.17pp. The **full-quality** mode drift reflects the BF16→FP16 cast performed at weight loading: NestedFP encodes its base and residual tensors with respect to FP16, so deploying a BF16-native model requires one rounding step that introduces sub-pp drift. **performance** mode and **full-quality** mode score within $\pm 0.5\text{pp}$ of one another on average, but this does not mean **performance** mode can replace **full-quality** mode. Per-prompt pass@1 is largely insensitive to the precision difference between them, while **full-quality** mode’s defining property, producing the same outputs as the deployed FP16 model, is not captured by per-prompt scoring (Section V-B2 returns to this with online evidence). Static FP8 shows the largest variance. Mean Δ is modest

on Qwen (-0.15 pp) and GLM (-0.52 pp), but Phi-3.5-MoE drops -2.88 pp on average, driven by IFEval where the score falls from 64.88 to 42.70. Both DPS configurations avoid this collapse on Phi thanks to the partial FP16 retention inherited from NestedFP.

2) *Performance and Effective Pass@1*: Figure 6 reports the headline result on three models that span low to high KV pressure: Phi-3.5-MoE, Qwen3-30B-A3B, and GLM-4.7-Flash. We sweep the request rate from 0.5 to 8.5 req/s under Gamma-distributed arrivals at $CV = 2.0$, and we report five metrics: effective pass@1, SLO attainment, achieved RPS, mean TPOT, and mean TTFT. We discuss each model separately because the relative behavior of the seven serving methods changes qualitatively with KV pressure.

Phi-3.5-MoE: Low-Pressure Regime. Phi-3.5-MoE is underloaded over the tested rate range: KV cache never saturates, all seven methods hold $\geq 97\%$ SLO attainment up to 8.5 req/s, and effective pass@1 sits within a 19.1–21.0% envelope (2 pp wide) that reflects benchmark noise rather than method differentiation. DPS stays inside this envelope at every rate, showing that the SUM operational cost (CUDA VMM mappings, allocator state tracking, controller polling) imposes no measurable accuracy cost when the dynamic mechanism is dormant; This addresses the natural concern that DPS’s runtime infrastructure might tax simple workloads. Throughput tells the same story. Static FP8 leads, WEIGHT-OFFLOAD trails, and DPS, static FP16, and NestedFP stay within 15% of static FP8. In this regime DPS reduces to its base configuration, full-quality FP16 execution with no mode transitions, and tracks static FP16 on every operational metric. This also serves as a multi-GPU sanity check, since Phi-3.5-MoE requires $TP=2$ on H100: DPS’s per-rank shared regions and SUM Manager broadcasts add nothing on top of the tensor-parallel cost that static FP16 already pays.

Qwen3-30B-A3B: High-Pressure Regime. Qwen3-30B-A3B is the regime where Static FP16 fails most dramatically and DPS’s dynamic mechanism contributes most. At 1.0 req/s (below saturation), DPS, Static FP16, Static FP8, and NestedFP all deliver 56.7–58.0% effective pass@1, confirming that DPS preserves FP16-class quality at low load. As the rate increases, the curves diverge sharply: by 2.5 req/s, Static FP16 has collapsed to 31.8% effective pass@1 (from 56.8%), driven by SLO attainment falling to 46.2%, while DPS sustains 57.8% with full SLO compliance. By 4.5 req/s, DPS delivers 56.4% against Static FP16’s 15.2%, a $+41.2$ pp absolute advantage.

The Static FP16 collapse is not a quality failure: its unconditional pass@1 at 4.5 req/s remains $\sim 57.4\%$. The collapse stems from SLO violation, since Static FP16 has limited KV cache capacity, and under the pressure, a large amount of requests violate the SLO, contributing zero to the effective pass@1. NestedFP, KV-OFFLOAD, and KV-QUANT collapse similarly; KV-QUANT degrades more gracefully because per-token KV reduction expands effective capacity, but still trails DPS by 9–14 pp across the burst region. Static FP8 tracks DPS throughout (56.5% vs. 56.4% at 4.5 req/s), but pays its quality cost permanently rather than only during bursts.

Throughput and latency expose the mechanism. Static FP16 sustains only 0.95 RPS at 1.0 req/s before SLO collapses, whereas DPS sustains 3.11 RPS at 4.5 req/s, a $3.3\times$ extension of FP16’s viable range that matches Static FP8 (3.16 RPS) without paying FP8’s accuracy cost. NestedFP gains nothing over Static FP16 (0.94 RPS) because its GPU memory is used to store the dual-precision weight. TTFT tells the most dramatic story: Static FP16 climbs from 428 ms at 1.0 req/s to 8.4 s at 1.5 and 21.0 s at 2.0, while DPS stays at 88–93 ms up to 3.0 req/s (vs. 61 s for Static FP16, a $\sim 660\times$ reduction) and remains within the 2 s prefill threshold (0.97 s) even at its terminal SLO-feasible rate of 4.5 req/s. TPOT confirms a small per-token reconstruction cost: Static FP8 is fastest (13–24 ms) on its smaller weight footprint, while DPS in full-quality mode tracks Static FP16 within 1–3 ms (20–28 vs. 19–30 ms), bounded regardless of load.

GLM-4.7-Flash: Moderate-Pressure Regime. GLM-4.7-Flash repeats Qwen’s pattern shifted right on the rate axis, since its smaller per-request KV footprint slows the buildup of KV pressure. At 1.0 req/s, all methods are near-equivalent (DPS 35.3%, Static FP16 35.6%, Static FP8 33.9%). Static FP16, NestedFP, and KV-OFFLOAD start failing the SLO around 2.5 req/s, and their effective pass@1 falls from 35% to 25% by 4.5 req/s purely from SLO violation (unconditional pass@1 stays near 35%). DPS sustains 35.2% at 4.5 req/s with full SLO compliance: a $+9.5$ pp gain over Static FP16, and $+1.3$ pp over Static FP8 (33.9%) because DPS spends part of the low-load intervals in full-quality mode. Throughput follows the same pattern: Static FP16 caps at 1.80 RPS at 2.2 req/s while DPS sustains 3.78 RPS at 6.0 req/s, a $2.1\times$ extension. NestedFP again matches Static FP16 (1.34 RPS) for the same structural reason as on Qwen.

On the latency side, mean TTFT for static FP16 climbs from 910 ms at req/s of 2.0 to 10.3 s at 3.0, while DPS holds at 130–147 ms up to req/s of 4.5 (vs. 24 s for static FP16, a roughly $165\times$ reduction). DPS’s mean TTFT remains within the 2-second prefill threshold throughout its operating range (0.46 s at req/s of 6.0), while FP16 has long since left it. TPOT splits the methods into three tiers: Static FP8 is fastest (14–26 ms), DPS sits slightly higher due to its reconstruction cost (22–30 ms), and Static FP16 is slowest (21–35 ms). The offloading baselines degrade much more sharply under load: KV-OFFLOAD climbs to 71 ms because every decode step fetches KV blocks from CPU memory, and WEIGHT-OFFLOAD stays permanently above 60 ms because every forward pass touches a fraction of host-resident weights. DPS’s background restore stays off the decode hot path: residual transfers run on a separate copy stream and mode transitions are batch-aligned (Section IV-D).

Impact of full-quality mode. Table I shows **full-quality** mode and **performance** mode within noise offline, raising the question of **full-quality** mode’s contribution. To isolate per-request output quality, we compare DPS against static FP8 only where both clear $\geq 95\%$ SLO attainment, so neither is dropping requests. In this regime, DPS’s unconditional pass@1 exceeds static FP8’s by $+1.39$ pp on GLM, $+0.46$ pp on Qwen,

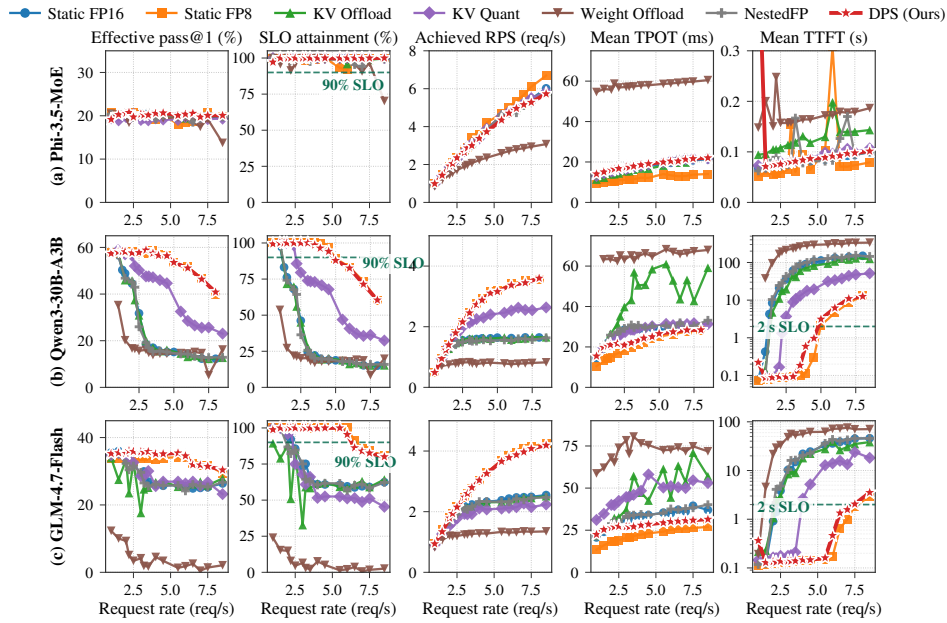


Fig. 6: Effective pass@1, SLO attainment, and mean TTFT vs. request rate on H100 across three MoE models. The 2 s SLO line is omitted from (a) because all systems remain well below the threshold.

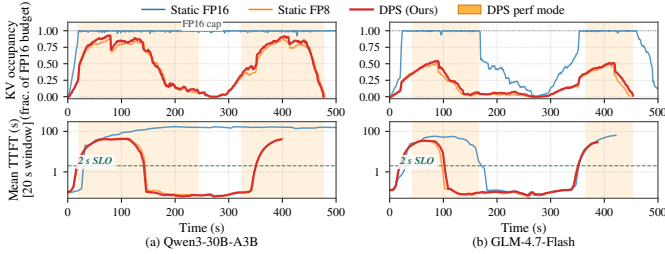


Fig. 7: KV cache occupancy (top) and mean TTFT (bottom) of DPS on (a) Qwen3-30B-A3B and (b) GLM-4.7-Flash.

and +0.32 pp on Phi, winning on 30 of 35 (model, req/s) combinations. Additionally, on Qwen, DPS’s online unconditional pass@1 (~58%) closely matches **performance** mode’s offline LiveCodeBench score (57.25%), so the win over static FP8 is explained by **performance** mode’s FP16 retention alone. On GLM, DPS’s online score (35.2–35.7%) clearly exceeds **performance** mode’s offline score (35.07%), and the online gap over static FP8 (+1.39 pp) is substantially larger than the offline **performance** mode-vs-FP8 gap (+0.57 pp). Therefore, the residual ~0.8–1.0 pp on GLM reflects the contribution of **full-quality** mode in the online scenario. The rate trend confirms it: the gap is largest at low rates (+1.83 pp at $\lambda = 1.0$) where DPS spends most time in **full-quality** mode, and shrinks to zero at high rates (−0.03 pp at $\lambda = 7.5$) as KV pressure forces the system to switch to **performance** mode. **Summary.** Across the three regimes, a consistent story emerges. At low load, DPS pays a few-millisecond TPOT overhead and at most 25 ms TTFT overhead for SUM book-keeping and FP16 reconstruction. Once KV pressure binds, DPS delivers 2.1–3.3 $\times$ higher sustained throughput, two-

orders-of-magnitude TTFT improvements over Static FP16, and better effective pass@1. Under KV pressure, the elastic KV capacity of DPS reduce the preemption, lets DPS keep larger active batches and amortize the per-step weight read, while Static FP16’s fixed budget cannot hold the batch full. Static FP8 is the only baseline matching DPS on throughput and SLO compliance, but DPS still gains 1–3 pp effective pass@1 on GLM (from time spent in full-quality mode at low and moderate rates) at a small TPOT cost. No single static configuration wins across regimes: Static FP8 wins TPOT at the cost of permanent accuracy loss, Static FP16 wins TPOT at low load but collapses under bursts, and DPS wins on effective pass@1 across the entire range.

3) *DPS Characterization:* This section traces the internal state behind those outcomes during a representative window, run at 4.5 req/s on Qwen3-30B-A3B and 6.0 req/s on GLM-4.7-Flash — rates where Static FP16 has collapsed but DPS sustains the SLO. Figure 7 reports two signals on a shared time axis: KV cache occupancy (top) and mean TTFT (bottom).

KV occupancy. Static FP16 stays at the FP16 budget for nearly the whole window. Once the cap is reached, every burst arrival must queue or preempt. DPS avoids this by reclaiming residual-weight units: KV occupancy oscillates between 0 and 0.93 on Qwen and 0 and 0.54 on GLM, with peaks aligned to bursts. The amber bands show DPS enters performance mode exactly when KV demand exceeds the FP16 budget — about 79.2% of the trace on Qwen, 47.1% on GLM — and reverts to full-quality mode when pressure subsides. DPS evicts up to 40 of 40 residual units on Qwen (the entire shared region) and 14 of 40 on GLM, reflecting GLM’s smaller per-request KV footprint.

TTFT. Static FP16’s mean TTFT rises up to 170 s on Qwen

TABLE II: Mode switching overhead under BurstGPT window’s traces. Restore latency is the per-cycle mean of the weight H2D wall time.

Model	Time in mode (%)		Transitions		Restore
	Full	Perf	F→P	P→F	Latency (ms)
Phi-3.5-MoE	100.0	0.0	0	0	—
Qwen3-30B-A3B	20.8	79.2	2	2	24651
GLM-4.7-Flash	52.9	47.1	2	2	8497

and 60 s on GLM — two orders of magnitude above the 2 s SLO, which is caused by the KV pressure. In two models, DPS crosses the SLO only at the trace start and end; during steady serving, TTFT remains below the SLO. Static allocation turns a transient memory spike into a long throughput collapse, since preemption cannot enlarge the budget. DPS turns the same spike into a brief precision excursion, with capacity restored once the burst passes. The $2.7\text{--}4.5\times$ operating-range extension and up to +41 pp effective pass@1 reported in Section V-B2 follow directly from this observation.

4) *Mode Switching Overhead:* Table II summarizes mode-switching activity under the bursty traces of Section V-B2. DPS spends 100% of serving time in full-quality mode on Phi-3.5-MoE (no KV pressure, no transitions), matching the no-overhead behavior in Figure 6(a). On Qwen3-30B-A3B and GLM-4.7-Flash, only two full → perf and two perf → full transitions occur across the entire trace, showing that the threshold-with-hysteresis policy (Section IV-D) consolidates bursts into a few long mode-windows rather than flapping rapidly. The reverse transition dominates the cost: average end-to-end restore latency is 8.5 s on GLM and 24.7 s on Qwen, set by the H2D transfer of 14/40 and 40/40 residual units from pinned CPU memory at near-identical per-unit cost (607 vs. 616 ms/unit) — restoration thus scales linearly with evicted units rather than total model size, and GLM’s lower total reflects only its lighter peak pressure. Because this transfer runs on the background restore worker and overlaps with ongoing decode, it does not block requests; its only runtime cost is bounded PCIe contention, and the user-visible impact remains negligible (Section V-B2).

VI. RELATED WORKS AND DISCUSSION

LLM Serving Systems with Virtual Memory. One fundamental feature of DPS is to preserve paged KV cache management in modern LLM-serving systems, such as in vLLM [9], [10]. Some recent approaches, such as [55], [54], also leverage dynamic memory management when serving multiple LLM models. [55] primarily targets memory sharing across different vLLM engines, while [54] focuses on serving with a quantized model and an FP16 model.

Model Offloading. DPS performs offloading of partial model state from CPU to GPU during full-quality-mode restoration. There are many recent offloading techniques, such as those in [56], [57], [54], for addressing GPU memory constraints in LLM serving systems. A common concept is to hide host-to-device memory transfers during GPU computation. For

example, [56] executes a layer in batches, providing sufficient GPU computation time to hide model transfers. [54] assumes that model transfers can be hidden by GPU computation.

Future Work. Page-based KV memory management may incur memory fragmentation [58], [59]. Meanwhile, the nested FP8-FP16 model is a specific case of dynamic networks. Investigating KV cache fragmentation or extending the method to various dynamic networks are promising future directions for DPS with SUM.

VII. CONCLUSION

This paper introduces an effective dual-precision serving with semi-unified memory. The proposed scheme is built on two fundamental concepts: (1) SUM to effectively mitigate KV cache pressure and possibly reduce preemptions, and (2) dual-precision serving with asymmetric switching based on nested models. With SUM, it turns weight memory into an elastic resource: under normal load, DPS serves the higher-precision model; under KV pressure, it switches to a nested, lower-precision variant and repurposes unused weight memory for KV cache blocks. DPS can serve as an extension for existing LLM serving systems, such as vLLM [10] while maintaining the effective dynamic structure of KV cache management.

REFERENCES

- [1] T. Brown, B. Mann, N. Ryder *et al.*, “Language models are few-shot learners,” in *Advances in Neural Information Processing Systems*, vol. 33, 2020.
- [2] H. Touvron, L. Martin, K. Stone, P. Albert *et al.*, “Llama 2: Open foundation and fine-tuned chat models,” 2023.
- [3] J. Bai, S. Bai *et al.*, “Qwen technical report,” 2023.
- [4] DeepSeek-AI, A. Liu, B. Feng, B. Wang, B. Wang, B. Liu, C. Zhao, C. Deng, C. Ruan, D. Dai, D. Guo, D. Yang, D. Chen, D. Ji, E. Li, F. Lin, F. Luo, G. Hao, G. Chen, G. Li, H. Zhang, H. Xu, H. Yang, H. Zhang, H. Ding, H. Xin, H. Gao, H. Li, H. Qu, J. L. Cai, J. Liang, J. Guo, J. Ni, J. Li, J. Chen, J. Yuan, J. Qiu, J. Song, K. Dong, K. Gao, K. Sun, K. Guan, L. Wang, L. Zhang, L. Xu, L. Xia, L. Zhao, L. Zhang, M. Li, M. Wang, M. Zhang, M. Zhang, M. Tang, M. Li, N. Tian, P. Huang, P. Wang, P. Zhang, Q. Zhu, Q. Chen, Q. Du, R. J. Chen, R. L. Jin, R. Ge, R. Pan, R. Xu, R. Chen, S. S. Li, S. Lu, S. Zhou, S. Chen, S. Wu, S. Ye, S. Ma, S. Wang, S. Zhou, S. Yu, S. Zhou, S. Zhou, S. Zheng, T. Wang, T. Pei, T. Yuan, T. Sun, W. L. Xiao, W. Zeng, W. An, W. Liu, W. Liang, W. Gao, W. Zhang, X. Q. Li, X. Jin, X. Wang, X. Bi, X. Liu, X. Wang, X. Shen, X. Chen, X. Chen, X. Nie, X. Sun, X. Wang, X. Liu, X. Xie, X. Yu, X. Song, X. Zhou, X. Yang, X. Lu, X. Su, Y. Wu, Y. K. Li, Y. X. Wei, Y. X. Zhu, Y. Xu, Y. Huang, Y. Li, Y. Zhao, Y. Sun, Y. Li, Y. Wang, Y. Zheng, Y. Zhang, Y. Xiong, Y. Zhao, Y. He, Y. Tang, Y. Piao, Y. Dong, Y. Tan, Y. Liu, Y. Wang, Y. Guo, Y. Zhu, Y. Wang, Y. Zou, Y. Zha, Y. Ma, Y. Yan, Y. You, Y. Liu, Z. Z. Ren, Z. Ren, Z. Sha, Z. Fu, Z. Huang, Z. Zhang, Z. Xie, Z. Hao, Z. Shao, Z. Wen, Z. Xu, Z. Zhang, Z. Li, Z. Wang, Z. Gu, Z. Li, and Z. Xie, “Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model,” 2024. [Online]. Available: <https://arxiv.org/abs/2405.04434>
- [5] DeepSeek-AI, A. Liu, B. Feng, B. Xue, B. Wang, B. Wu, C. Lu, C. Zhao, C. Deng, C. Zhang, C. Ruan, D. Dai, D. Guo, D. Yang, D. Chen, D. Ji, E. Li, F. Lin, F. Dai, F. Luo, G. Hao, G. Chen, G. Li, H. Zhang, H. Bao, H. Xu, H. Wang, H. Zhang, H. Ding, H. Xin, H. Gao, H. Li, H. Qu, J. L. Cai, J. Liang, J. Guo, J. Ni, J. Li, J. Wang, J. Chen, J. Chen, J. Yuan, J. Qiu, J. Li, J. Song, K. Dong, K. Hu, K. Gao, K. Guan, K. Huang, K. Yu, L. Wang, L. Zhang, L. Xu, L. Xia, L. Zhao, L. Wang, L. Zhang, M. Li, M. Wang, M. Zhang, M. Zhang, M. Tang, M. Li, N. Tian, P. Huang, P. Wang, P. Zhang, Q. Wang, Q. Zhu, Q. Chen, Q. Du, R. J. Chen, R. L. Jin, R. Ge, R. Zhang, R. Pan, R. Wang, R. Xu, R. Zhang, R. Chen, S. S. Li, S. Lu, S. Zhou, S. Chen, S. Wu, S. Ye, S. Ye, S. Ma, S. Wang, S. Zhou, S. Yu, S. Zhou,

- S. Pan, T. Wang, T. Yun, T. Pei, T. Sun, W. L. Xiao, W. Zeng, W. Zhao, W. An, W. Liu, W. Liang, W. Gao, W. Yu, W. Zhang, X. Q. Li, X. Jin, X. Wang, X. Bi, X. Liu, X. Wang, X. Shen, X. Chen, X. Zhang, X. Chen, X. Nie, X. Sun, X. Wang, X. Cheng, X. Liu, X. Xie, X. Liu, X. Yu, X. Song, X. Shan, X. Zhou, X. Yang, X. Li, X. Su, X. Lin, Y. K. Li, Y. Q. Wang, Y. X. Wei, Y. X. Zhu, Y. Zhang, Y. Xu, Y. Xu, Y. Huang, Y. Li, Y. Zhao, Y. Sun, Y. Li, Y. Wang, Y. Yu, Y. Zheng, Y. Zhang, Y. Shi, Y. Xiong, Y. He, Y. Tang, Y. Piao, Y. Wang, Y. Tan, Y. Ma, Y. Liu, Y. Guo, Y. Wu, Y. Ou, Y. Zhu, Y. Wang, Y. Gong, Y. Zou, Y. He, Y. Zha, Y. Xiong, Y. Ma, Y. Yan, Y. Luo, Y. You, Y. Liu, Y. Zhou, Z. F. Wu, Z. Z. Ren, Z. Ren, Z. Sha, Z. Fu, Z. Xu, Z. Huang, Z. Zhang, Z. Xie, Z. Zhang, Z. Hao, Z. Gou, Z. Ma, Z. Yan, Z. Shao, Z. Xu, Z. Wu, Z. Zhang, Z. Li, Z. Gu, Z. Zhu, Z. Liu, Z. Li, Z. Xie, Z. Song, Z. Gao, and Z. Pan, "Deepseek-v3 technical report," 2025. [Online]. Available: <https://arxiv.org/abs/2412.19437>
- [6] OpenAI, "Chatgpt," 2024. [Online]. Available: <https://chat.openai.com>
- [7] Microsoft, "Github copilot," 2024. [Online]. Available: <https://github.com/features/copilot>
- [8] G.-I. Yu, J. S. Jeong, G.-W. Kim, S. Kim, and B.-G. Chun, "Orca: A distributed serving system for Transformer-Based generative models," in *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI '22)*. Carlsbad, CA: USENIX Association, Jul. 2022, pp. 521–538. [Online]. Available: <https://www.usenix.org/conference/osdi22/presentation/you>
- [9] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, "Efficient memory management for large language model serving with pagedattention," 2023. [Online]. Available: <https://arxiv.org/abs/2309.06180>
- [10] —, "Efficient memory management for large language model serving with pagedattention," in *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP '23)*. ACM, 2023, pp. 611–626. [Online]. Available: <https://doi.org/10.1145/3600006.3613165>
- [11] L. Zheng, L. Yin, Z. Xie, C. Sun, J. Huang, C. H. Yu, S. Cao, C. Kozyrakis, I. Stoica, J. E. Gonzalez, C. W. Barrett, and Y. Sheng, "Sglang: Efficient execution of structured language model programs," 2023. [Online]. Available: <https://arxiv.org/abs/2312.07104>
- [12] —, "Sglang: Efficient execution of structured language model programs," in *Proceedings of the 38th Conference on Neural Information Processing Systems (NeurIPS 2024)*, 2024. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2024/file/724be4472168f31ba1c9ac630f15dec8-Paper-Conference.pdf
- [13] A. Yang, A. Li, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu, C. Gao, C. Huang, C. Lv, C. Zheng, D. Liu, F. Zhou, F. Huang, F. Hu, H. Ge, H. Wei, H. Lin, J. Tang, J. Yang, J. Tu, J. Zhang, J. Yang, J. Yang, J. Zhou, J. Zhou, J. Lin, K. Dang, K. Bao, K. Yang, L. Yu, L. Deng, M. Li, M. Xue, M. Li, P. Zhang, P. Wang, Q. Zhu, R. Men, R. Gao, S. Liu, S. Luo, T. Li, T. Tang, W. Yin, X. Ren, X. Wang, X. Zhang, X. Ren, Y. Fan, Y. Su, Y. Zhang, Y. Zhang, Y. Wan, Y. Liu, Z. Wang, Z. Cui, Z. Zhang, Z. Zhou, and Z. Qiu, "Qwen3 technical report," 2025. [Online]. Available: <https://arxiv.org/abs/2505.09388>
- [14] M. Abidin, S. A. Jacobs, A. A. Awan, J. Aneja, A. Awadallah, H. Awadalla, N. Bach, A. Bahree, A. Bakhtiari, H. Behl *et al.*, "Phi-3 technical report: A highly capable language model locally on your phone," *arXiv preprint arXiv:2404.14219*, 2024.
- [15] Nvidia, "Nvidia h100 gpu," 2024. [Online]. Available: <https://www.nvidia.com/en-us/data-center/h100/>
- [16] —, "Nvidia v100 gpu," 2018. [Online]. Available: <https://www.nvidia.com/en-us/data-center/v100/>
- [17] —, "Nvidia a100 gpu," 2022. [Online]. Available: <https://www.nvidia.com/en-us/data-center/a100/>
- [18] N. P. Jouppi, C. Young, N. Patil, D. Patterson, G. Agrawal, R. Bajwa, S. Bates, S. Bhatia, N. Boden, A. Borchers, R. Boyle, P.-I. Cantin, C. Chao, C. Clark, J. Coriell, M. Daley, M. Dau, J. Dean, B. Gelb, T. V. Ghaemmaghami, R. Gottipati, W. Gulland, R. Hagmann, C. R. Ho, D. Hogberg, J. Hu, R. Hundt, D. Hurt, J. Ibarz, A. Jaffey, A. Jaworski, A. Kaplan, H. Khaitan, D. Killebrew, A. Koch, N. Kumar, S. Lacy, J. Laudon, J. Law, D. Le, C. Leary, Z. Liu, K. Lucke, A. Lundin, G. MacKean, A. Maggiore, M. Mahony, K. Miller, R. Nagarajan, R. Narayanaswami, R. Ni, K. Nix, T. Norrie, M. Omernick, N. Penukonda, A. Phelps, J. Ross, M. Ross, A. Salek, E. Samadiani, C. Severn, G. Sizikov, M. Snellman, J. Souter, D. Steinberg, A. Swing, M. Tan, G. Thorson, B. Tian, H. Toma, E. Tuttle, V. Vasudevan, R. Walter, W. Wang, E. Wilcox, and D. H. Yoon, "In-datacenter performance analysis of a tensor processing unit," *SIGARCH Comput. Archit. News*, vol. 45, no. 2, p. 1–12, Jun. 2017. [Online]. Available: <https://doi.org/10.1145/3140659.3080246>
- [19] N. Jouppi, G. Kurian, S. Li, P. Ma, R. Nagarajan, L. Nai, N. Patil, S. Subramanian, A. Swing, B. Towles, C. Young, X. Zhou, Z. Zhou, and D. A. Patterson, "Tpu v4: An optically reconfigurable supercomputer for machine learning with hardware support for embeddings," in *Proceedings of the 50th Annual International Symposium on Computer Architecture*, ser. ISCA '23. New York, NY, USA: Association for Computing Machinery, 2023. [Online]. Available: <https://doi.org/10.1145/3579371.3589350>
- [20] D. Thomas, "Benchmarking text generation inference," May 29, 2024. [Online]. Available: <https://huggingface.co/blog/tgi-benchmarking>
- [21] A. Kuzmin, M. Van Baalen, Y. Ren, M. Nagel, J. Peters, and T. Blankevoort, "Fp8 quantization: the power of the exponent," in *Proceedings of the 36th International Conference on Neural Information Processing Systems*, ser. NIPS '22. Red Hook, NY, USA: Curran Associates Inc., 2022.
- [22] J. Kim, J. Lee, G. Park, B. Kim, S. J. Kwon, D. Lee, and Y. Lee, "An inquiry into datacenter tco for llm inference with fp8," 2025. [Online]. Available: <https://arxiv.org/abs/2502.01070>
- [23] H. Shen, N. Mellempudi, X. He, Q. Gao, C. Wang, and M. Wang, "Efficient post-training quantization with fp8 formats," 2024. [Online]. Available: <https://arxiv.org/abs/2309.14592>
- [24] H. Lee, O. Kwon, Y. Park, and J. W. Lee, "Nestedfp: High-performance, memory-efficient dual-precision floating point support for llms," 2026. [Online]. Available: <https://arxiv.org/abs/2506.02024>
- [25] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar *et al.*, "Llama: Open and efficient foundation language models," *arXiv preprint arXiv:2302.13971*, 2023.
- [26] S. Rajbhandari, C. Li, Z. Yao, M. Zhang, R. Y. Aminabadi, A. A. Awan, J. Rasley, and Y. He, "DeepSpeed-MoE: Advancing mixture-of-experts inference and training to power next-generation ai scale," in *International conference on machine learning*. PMLR, 2022, pp. 18 332–18 346.
- [27] G. Xiao, J. Lin, M. Seznec, H. Wu, J. Demouth, and S. Han, "SmoothQuant: Accurate and efficient post-training quantization for large language models," in *International Conference on Machine Learning*. PMLR, 2023, pp. 38 087–38 099.
- [28] J. Lin, J. Tang, H. Tang, S. Yang, W.-M. Chen, W.-C. Wang, G. Xiao, X. Dang, C. Gan, and S. Han, "AWQ: Activation-aware Weight Quantization for On-Device LLM Compression and Acceleration," *Proceedings of Machine Learning and Systems*, vol. 6, pp. 87–100, 2024.
- [29] Y. Lin, H. Tang, S. Yang, Z. Zhang, G. Xiao, C. Gan, and S. Han, "QServe: W4A8KV4 quantization and system co-design for efficient LLM serving," in *Eighth Conference on Machine Learning and Systems*, 2025.
- [30] Y. Zhang, P. Zhang, M. Huang, J. Xiang, Y. Wang, C. Wang, Y. Zhang, L. Yu, C. Liu, and W. Lin, "QQQ: Quality Quattuor-Bit Quantization for Large Language Models," *arXiv preprint arXiv:2406.09904*, 2024.
- [31] S. M. Lee, H. Kim, J. Yeon, M. Kim, C. Park, B. Bae, Y. Cha, W. Choe, J. Choi, Y. Choi, K. J. Han, S. Hwang, K. Jang, J. Jeon, H. Jeong, Y. Jung, H. Kim, S. Kim, S. Kim, W. Kim, Y. Kim, Y. Kim, H. Kwon, J. K. Lee, J. Lee, K. Lee, S. Lee, M. Noh, J. Park, J. Seo, and J. Paik, "16.2 RRGD: A 16nm tensor-contraction processor for power-efficient inference on large language models," in *IEEE International Solid-State Circuits Conference, ISSCC 2025, San Francisco, CA, USA, February 16-20, 2025*. IEEE, 2025, pp. 284–286. [Online]. Available: <https://doi.org/10.1109/ISSCC49661.2025.10904727>
- [32] ShareGPT, "Sharegpt," 2023. [Online]. Available: <https://sharegpt.com>
- [33] Y. Wang, Y. Chen, Z. Li, X. Kang, Y. Fang, Y. Zhou, Y. Zheng, Z. Tang, X. He, R. Guo, X. Wang, Q. Wang, A. C. Zhou, and X. Chu, "Burstgpt: A real-world workload dataset to optimize llm serving systems," in *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V2*, ser. KDD '25. New York, NY, USA: Association for Computing Machinery, 2025, p. 5831–5841. [Online]. Available: <https://doi.org/10.1145/3711896.3737413>
- [34] S. Rando, L. Romani, A. Sampieri, L. Franco, J. Yang, Y. Kyuragi, F. Galasso, and T. Hashimoto, "Longcodebench: Evaluating coding llms at 1m context windows," 2025. [Online]. Available: <https://arxiv.org/abs/2505.07897>
- [35] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, "Swe-bench: Can language models resolve real-world

- github issues?” 2024. [Online]. Available: <https://arxiv.org/abs/2310.06770>
- [36] R. Cao, M. Chen, J. Chen, Z. Cui, Y. Feng, B. Hui, Y. Jing, K. Li, M. Li, J. Lin, Z. Ma, K. Shum, X. Wang, J. Wei, J. Yang, J. Zhang, L. Zhang, Z. Zhang, W. Zhao, and F. Zhou, “Qwen3-coder-next technical report,” 2026. [Online]. Available: <https://arxiv.org/abs/2603.00729>
- [37] H. Yu, H. Li, H. Shi, T. S. Huang, and G. Hua, “Any-precision deep neural networks,” *CoRR*, vol. abs/1911.07346, 2019. [Online]. Available: <http://arxiv.org/abs/1911.07346>
- [38] Y. Park, J. Hyun, S. Cho, B. Sim, and J. W. Lee, “Any-precision llm: low-cost deployment of multiple, different-sized llms,” in *Proceedings of the 41st International Conference on Machine Learning*, ser. ICML’24. JMLR.org, 2024.
- [39] J.-Y. Jeon, X. T. Nguyen, S. Ryu, and H.-J. Lee, “Usdn: A unified sample-wise dynamic network with mixed-precision and early-exit,” in *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, January 2024, pp. 646–654.
- [40] Y. Chen, X. Pan, Y. Li, B. Ding, and J. Zhou, “Ee-llm: large-scale training and inference of early-exit large language models with 3d parallelism,” in *Proceedings of the 41st International Conference on Machine Learning*, ser. ICML’24. JMLR.org, 2024.
- [41] J. Xu, J. Pan, Y. Zhou, S. Chen, J. Li, Y. Lian, J. Wu, and G. Dai, “Speccee: Accelerating large language model inference with speculative early exiting,” in *Proceedings of the 52nd Annual International Symposium on Computer Architecture*, ser. ISCA ’25. New York, NY, USA: Association for Computing Machinery, 2025, p. 467–481. [Online]. Available: <https://doi.org/10.1145/3695053.3730996>
- [42] Nvidia, “Virtual memory managment,” May 29, 2024. [Online]. Available: https://docs.nvidia.com/cuda/cuda-driver-api/group__CUDA__VA.html
- [43] —, “Virtual memory managment,” May 29, 2024. [Online]. Available: <https://docs.nvidia.com/cuda/cuda-programming-guide/04-special-topics/virtual-memory-management.html>
- [44] M. Kleinegger, E. Crnčević, and D. Alistarh, “Matgptq: Accurate and efficient post-training matryoshka quantization,” 2026. [Online]. Available: <https://arxiv.org/abs/2602.03537>
- [45] M. L. Puterman, *Markov decision processes: discrete stochastic dynamic programming*. John Wiley & Sons, 2014.
- [46] S. Kakade and J. Langford, “Approximately optimal approximate reinforcement learning,” in *Proceedings of the nineteenth international conference on machine learning*, 2002, pp. 267–274.
- [47] M. Abdin, J. Aneja, H. Awadalla, A. Awadallah, A. A. Awan, N. Bach, A. Bahree, A. Bakhtiari, J. Bao, H. Behl, A. Benhaim, M. Bilenko, J. Bjorck, S. Bubeck, M. Cai, Q. Cai, V. Chaudhary, D. Chen, D. Chen, W. Chen, Y.-C. Chen, Y.-L. Chen, H. Cheng, P. Chopra, X. Dai, M. Dixon, R. Eldan, V. Fragoso, J. Gao, M. Gao, M. Gao, A. Garg, A. D. Giorno, A. Goswami, S. Gunasekar, E. Haider, J. Hao, R. J. Hewett, W. Hu, J. Huynh, D. Iter, S. A. Jacobs, M. Javaheripi, X. Jin, N. Karampatziakis, P. Kauffmann, M. Khademi, D. Kim, Y. J. Kim, L. Kurilenko, J. R. Lee, Y. T. Lee, Y. Li, Y. Li, C. Liang, L. Liden, X. Lin, Z. Lin, C. Liu, L. Liu, M. Liu, W. Liu, X. Liu, C. Luo, P. Madan, A. Mahmoudzadeh, D. Majercak, M. Mazzola, C. C. T. Mendes, A. Mitra, H. Modi, A. Nguyen, B. Norick, B. Patra, D. Perez-Becker, T. Portet, R. Pryzant, H. Qin, M. Radmilac, L. Ren, G. de Rosa, C. Rosset, S. Roy, O. Ruwase, O. Saarikivi, A. Saied, A. Salim, M. Santacrose, S. Shah, N. Shang, H. Sharma, Y. Shen, S. Shukla, X. Song, M. Tanaka, A. Tupini, P. Vaddamanu, C. Wang, G. Wang, L. Wang, S. Wang, X. Wang, Y. Wang, R. Ward, W. Wen, P. Witte, H. Wu, X. Wu, M. Wyatt, B. Xiao, C. Xu, J. Xu, W. Xu, J. Xue, S. Yadav, F. Yang, J. Yang, Y. Yang, Z. Yang, D. Yu, L. Yuan, C. Zhang, C. Zhang, J. Zhang, L. L. Zhang, Y. Zhang, Y. Zhang, Y. Zhang, and X. Zhou, “Phi-3 technical report: A highly capable language model locally on your phone,” 2024. [Online]. Available: <https://arxiv.org/abs/2404.14219>
- [48] . Team, A. Zeng, X. Lv, Q. Zheng, Z. Hou, B. Chen, C. Xie, C. Wang, D. Yin, H. Zeng, J. Zhang, K. Wang, L. Zhong, M. Liu, R. Lu, S. Cao, X. Zhang, X. Huang, Y. Wei, Y. Cheng, Y. An, Y. Niu, Y. Wen, Y. Bai, Z. Du, Z. Wang, Z. Zhu, B. Zhang, B. Wen, B. Wu, B. Xu, C. Huang, C. Zhao, C. Cai, C. Yu, C. Li, C. Ge, C. Huang, C. Zhang, C. Xu, C. Zhu, C. Li, C. Yin, D. Lin, D. Yang, D. Jiang, D. Ai, E. Zhu, F. Wang, G. Pan, G. Wang, H. Sun, H. Li, H. Li, H. Hu, H. Zhang, H. Peng, H. Tai, H. Zhang, H. Wang, H. Yang, H. Liu, H. Zhao, H. Liu, H. Yan, H. Liu, H. Chen, J. Li, J. Zhao, J. Ren, J. Jiao, J. Zhao, J. Yan, J. Wang, J. Gui, J. Zhao, J. Liu, J. Li, J. Li, J. Lu, J. Wang, J. Yuan, J. Li, J. Du, J. Du, J. Liu, J. Zhi, J. Gao, K. Wang, L. Yang, L. Xu, L. Fan, L. Wu, L. Ding, L. Wang, M. Zhang, M. Li, M. Xu, M. Zhao, M. Zhai, P. Du, Q. Dong, S. Lei, S. Tu, S. Yang, S. Lu, S. Li, S. Li, Shuang-Li, S. Yang, S. Yi, T. Yu, W. Tian, W. Wang, W. Yu, W. L. Tam, W. Liang, W. Liu, X. Wang, X. Jia, X. Gu, X. Ling, X. Wang, X. Fan, X. Pan, X. Zhang, X. Zhang, X. Fu, X. Zhang, Y. Xu, Y. Wu, Y. Lu, Y. Wang, Y. Zhou, Y. Pan, Y. Zhang, Y. Wang, Y. Li, Y. Su, Y. Geng, Y. Zhu, Y. Yang, Y. Li, Y. Wu, Y. Li, Y. Liu, Y. Wang, Y. Li, Y. Zhang, Z. Liu, Z. Yang, Z. Zhou, Z. Qiao, Z. Feng, Z. Liu, Z. Zhang, Z. Wang, Z. Yao, Z. Wang, Z. Liu, Z. Chai, Z. Li, Z. Zhao, W. Chen, J. Zhai, B. Xu, M. Huang, H. Wang, J. Li, Y. Dong, and J. Tang, “Glm-4.5: Agentic, reasoning, and coding (arc) foundation models,” 2025. [Online]. Available: <https://arxiv.org/abs/2508.06471>
- [49] vLLM Project, “FP8 W8A8 quantization in vLLM,” <https://docs.vllm.ai/en/stable/features/quantization/fp8/>, 2024, accessed: 2026-05-01.
- [50] A. Agrawal, N. Kedia, A. Panwar, J. Mohan, N. Kwatra, B. S. Gulavani, A. Tumanov, and R. Ramjee, “Taming throughput-latency tradeoff in LLM inference with Sarathi-Serve,” in *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. Santa Clara, CA: USENIX Association, Jul. 2024, pp. 117–134. [Online]. Available: <https://www.usenix.org/conference/osdi24/presentation/agrawal>
- [51] Y. Zhong, S. Liu, J. Chen, J. Hu, Y. Zhu, X. Liu, X. Jin, and H. Zhang, “DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving,” in *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, 2024, pp. 193–210.
- [52] S. Chen, Z. Jia, S. Khan, A. Krishnamurthy, and P. B. Gibbons, “SLOs-Serve: Optimized serving of multi-SLO LLMs,” *arXiv preprint arXiv:2504.08784*, 2025. [Online]. Available: <https://arxiv.org/abs/2504.08784>
- [53] Z. Wang, S. Li, Y. Zhou, X. Li, R. Gu, N. Cam-Tu, C. Tian, and S. Zhong, “Revisiting SLO and goodput metrics in LLM serving,” *arXiv preprint arXiv:2410.14257*, 2024. [Online]. Available: <https://arxiv.org/abs/2410.14257>
- [54] Z. Su, Z. Zhang, T. Lan, Z. Wang, H. Shen, J. Yang, and Y. Cheng, “Morphserve: Efficient and workload-aware llm serving via runtime quantized layer swapping and kv cache resizing,” 2026. [Online]. Available: <https://arxiv.org/abs/2506.02006>
- [55] S. Yu, J. Xing, Y. Qiao, M. Ma, Y. Li, Y. Wang, S. Yang, Z. Xie, S. Cao, K. Bao, I. Stoica, H. Xu, and Y. Sheng, “Prism: Unleashing gpu sharing for cost-efficient multi-llm serving,” 2025. [Online]. Available: <https://arxiv.org/abs/2505.04021>
- [56] Y. Sheng, L. Zheng, B. Yuan, Z. Li, M. Ryabinin, B. Chen, P. Liang, C. Ré, I. Stoica, and C. Zhang, “Flexgen: high-throughput generative inference of large language models with a single gpu,” in *Proceedings of the 40th International Conference on Machine Learning*, ser. ICML’23. JMLR.org, 2023.
- [57] S. Cao, S. Liu, T. Griggs, P. Schafhalter, X. Liu, Y. Sheng, J. E. Gonzalez, M. Zaharia, and I. Stoica, “Moe-lightning: High-throughput moe inference on memory-constrained gpus,” in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*, ser. ASPLOS ’25. New York, NY, USA: Association for Computing Machinery, 2025, p. 715–730. [Online]. Available: <https://doi.org/10.1145/3669940.3707267>
- [58] R. Prabhu, A. Nayak, J. Mohan, R. Ramjee, and A. Panwar, “vattention: Dynamic memory management for serving llms without pagedattention,” in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*, ser. ASPLOS ’25. New York, NY, USA: Association for Computing Machinery, 2025, p. 1133–1150. [Online]. Available: <https://doi.org/10.1145/3669940.3707256>
- [59] J. Xu, R. Zhang, Y. Xiong, C. Guo, Z. Liu, Y. Zhou, W. Hu, H. Wu, C. Shao, Z. Wang, Y. Yuan, J. Zhao, M. Guo, and J. Leng, “ellm: Elastic memory management framework for efficient llm serving,” 2025. [Online]. Available: <https://arxiv.org/abs/2506.15155>